

PRAKTIKUM INFORMATIK 2

M. Farmbauer, M. Niemetz

Einführung Objekte

Queue

März 2026, Version: 2.3

1 Einleitung

Beginnend mit dieser Aufgabe enthalten die Angaben meist Hinweise auf Themen, die im weiteren Verlauf der Aufgabe benötigt werden, die Ihnen aber eventuell nicht mehr oder noch nicht geläufig sind.

Sie sollten diese Themen unbedingt nachschlagen – entweder vor Beginn der Bearbeitung der Aufgabe oder spätestens, wenn das jeweilige Konzept in der Aufgabenbearbeitung benötigt wird. Ansonsten besteht die Gefahr, dass Sie viel Zeit verschwenden und ein suboptimales Lernergebnis erzielen.

Sie können auch gerne Fragen zu diesen Themen in der IN2-Vorlesung stellen.

Vorbereitungsthemen Folgende Auflistung von Themen, die im Laufe dieser Aufgabe benötigt werden, soll Ihnen also bei der Abarbeitung der Aufgabe helfen.

Dabei ist auch jeweils die Seite angegeben, auf dieses Thema erstmals benötigt wird.

Manche der Themen sind nur eine Wiederholung von (hoffentlich) bereits Bekanntem. Oft müssen Sie sich aber auch neue Sachverhalte erarbeiten.

Hinweis: Sollten Sie bei der Vorbereitung dieser Themen Schwierigkeiten haben, wenden Sie sich mit Ihren Fragen an Ihre Dozenten in Praktikum oder Vorlesung.

Vorbereitung

- ☐ ☕☕ **Verkettete Liste:** Lesen Sie nochmals nach, wie das Konzept der verketteten Liste funktioniert, das Sie in IN1 kennengelernt haben. 3
- ☐ ☕ **Pointer in C und ihre Verwendung:** Wiederholen Sie, wie Pointer arbeiten und wie man sie korrekt verwendet – insbesondere in Verbindung mit Strukturdatentypen. 3
- ☐ ☕ **Remote-Repository hinzufügen:** Lesen Sie nach, wie Sie ein neues Remote-Repository zum git-Projekt hinzufügen können. 5
- ☐ ☕☕ **Definition einer neuen Klasse:** Lesen Sie nach, wie Sie eine neue Klasse definieren können. 6
- ☐ ☕ **Implementierung von Konstruktoren:** Schlagen Sie nach, was Konstruktoren sind, wozu sie dienen und wie Sie Konstruktoren implementieren können. Insbesondere sollten Sie dabei die Initialisierersliste recherchieren. 6

- ☐ ☕ **Defaultwerte für Funktionsparameter:** Schlagen Sie nach, wie Sie für Parameter einer Funktion Defaultwerte (manchmal auch verwirrenderweise als „Standardparameter“ bezeichnet) spezifizieren können, so dass diese Parameter zu optional anzugebenden Parametern werden. 6
- ☐ ☕ **Implementierung der Methoden außerhalb der Klassendefinition:** Recherchieren Sie, wie Sie mit Hilfe des Namespace-Operators die Implementierung der Methoden einer Klasse von der Definition der Klasse im `class`-Statement trennen können. 7
- ☐ ☕ **Ausgabe auf `cout`:** Schlagen Sie nach, wie Sie in C++ Ausgaben auf die Konsole über die Verwendung des `ostream`-Objekts `cout` durchführen können. 7
- ☐ ☕ **Dynamisches Anlegen und Löschen von Objekten:** Klären Sie, wie Sie in C++ Objekte dynamisch zur Laufzeit mit `new` anlegen und mit `delete` löschen können. 12
- ☐ ☕ **Destruktoren:** Lesen Sie nach, was Destruktoren sind und wozu sie dienen. 12

Überblick In dieser Aufgabe verwenden Sie das bereits aus Informatik 1 bekannte Konzept der Verketteten Liste und setzen es in einer an Ihr Einstiegsniveau angepasst vereinfachten objektorientierten Lösung um.

Dabei werden Sie eine sogenannte Queue (Warteschlange) implementieren, die auch als FIFO (First In First Out) bekannt ist und in zahlreichen Algorithmen und Anwendungen benötigt wird. Queues werden häufig verwendet, um Dinge zwischenspeichern, bevor sie serialisiert abgearbeitet werden können: Arbeitsaufträge in einer Firma, Druckaufträge, Schreibvorgänge auf eine Festplatte, Benutzeranforderungen, Tasteneingaben, Übertragung von Datenpaketen über ein Netzwerk, usw.

Zur Erinnerung an das Konzept sehen Sie in Abbildung 1 eine schematische Zeichnung einer (doppelt) verketteten Liste. Diese besitzt Elemente, die einerseits die gewünschten Nutzdaten enthalten:

- Eine kleine Zeichenkette als Beschreibung
- Drei Integer-Werte als Koordinaten eines Punktes im dreidimensionalen Raum

Zudem werden noch Pointer zu Verkettung der Elemente benötigt: Im vorliegenden Fall der Queue ist es günstig, zwei solcher Pointer zu verwenden, die jeweils auf beide Nachbarelemente, das Vorgängerelement und das Nachfolgerelement verweisen.

Das erhöht zwar den Aufwand (Programmieraufwand, Speicherplatzbedarf und Ausführungszeit) der Listenverwaltung, aber dadurch wird es möglich, die Liste in beiden Richtungen zu durchlaufen. Damit ergeben

Recherche ☕☕

Verkettete Liste

Recherche ☕

Pointer in C und ihre Verwendung

sich an anderer Stelle große Einsparungen im Vergleich zur einfach verketteten Liste, weil diese immer vom Anfang her durchlaufen werden muss, zum Beispiel um das Vorgängerelement des aktuellen Elements zu finden.

Wichtig: Diese Aufgabe ist gemessen an den bisherigen Aufgaben, die Sie aus PIN1 gewohnt sind, komplex aber nicht schwierig. Daher ist es unbedingt empfehlenswert, die Angabe sorgfältig und konzentriert zu lesen und danach vorzugehen. Herumbasteln um zur Lösung zu kommen, ist keine gute Idee und birgt das hohe Risiko, dass die Aufgabe für sehr viel Frust bei der Bearbeitung sorgt und Sie sehr viel wertvolle Zeit verschwenden.

Gehen Sie immer systematisch vor: Überlegen Sie sich Ihre Vorgehensweise, setzen Sie diese um und prüfen Sie dann den Erfolg. Gegebenenfalls beginnt das Spiel dann wieder von vorne. Gehen Sie dabei in einfachen, kleinen Schritten vor.

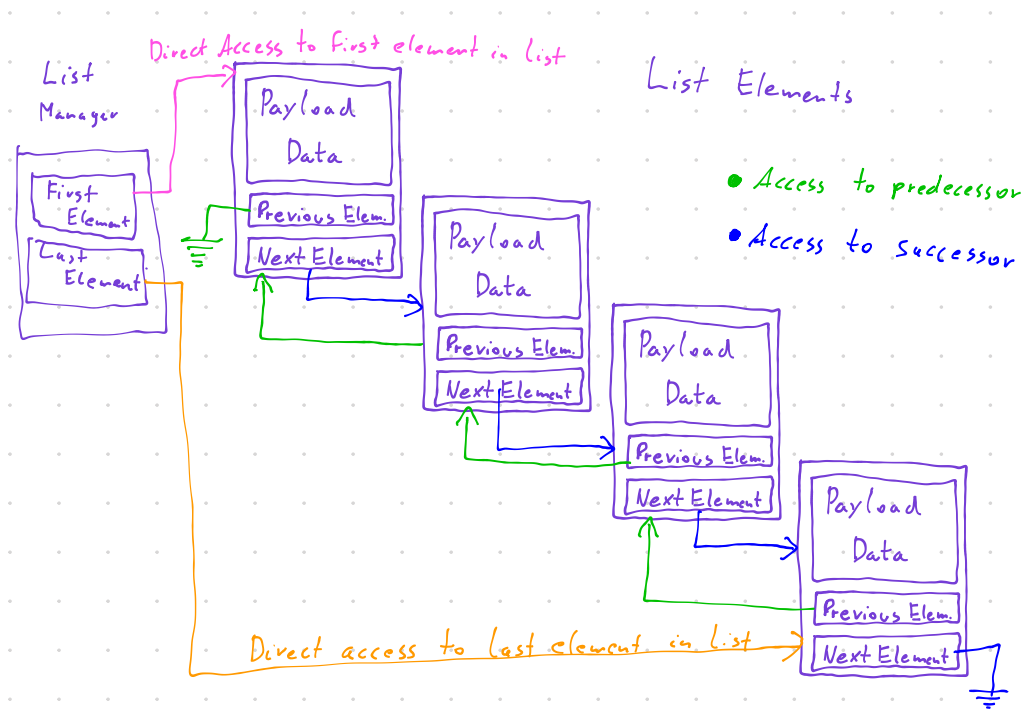


Abbildung 1: Diese Skizze zeigt eine schematische Darstellung einer doppelt verketteten Liste. Links erkennt man den Listenmanager, der einen Pointer auf den Listenanfang und auf das Listenende verwaltet, um auf beides direkt zugreifen zu können. Daneben findet man die Listenelemente, welche die Nutzdaten transportieren und durch Pointer mit ihrem Vorgängern und Nachfolgern verbunden sind. Die Methoden zur Verwaltung der Liste sind hier nicht dargestellt.

2 Aufgabenstellung

Zur Implementierung der Queue als verkettete Liste wird in mehreren Schritten vorgegangen.

Zunächst wird eine Klasse für ein einzelnes Element der doppelt verketteten Liste definiert und entsprechende Elementfunktionen zum Umgang mit diesen Datenelementen implementiert. Diese werden dann gründlich getestet.

Danach wird eine Klasse implementiert, welche die Liste verwaltet. Diese Klasse verwendet die zuvor entwickelte Klasse der Listenelemente um die Datenstruktur der Queue aufzubauen.

2.1 Aktualisieren des Projektes



Konfigurieren Sie in Ihrem Projekt das von Ihren Betreuern gepflegte Upstream-Projekt

EI/LV/PIN2/pin2_s2026

als zusätzliches Remote-Projekt namens **upstream**, falls Sie das noch nicht erledigt haben.

Die Anleitung für diesen Arbeitsschritt finden Sie in der separaten git-Anleitung.

Die erforderliche URL finden Sie wie in der vorangehenden Aufgabe über die Weboberfläche des Servers.



Aktualisieren Sie ihren Fork des PIN2-Projekts indem Sie `git pull upstream master` aufrufen.

Überprüfen Sie anschließend, ob sich das Verzeichnis für diese Aufgabe nun in Ihrem Repository befindet.

Hinweis: Das Projekt ist in diesem Zustand noch nicht lauffähig!



Sie können nun auch Ihr eigenes Projekt auf dem Server durch einen push-Vorgang wie gewohnt aktualisieren.



Aktualisieren Sie Ihren Fork des PIN2-Projekts.

Recherche ☕

Remote-Repository hinzufügen

2.2 Erstellen der Klasse QueueElement

Um die Definition der Listenelemente später auch in einem anderen Kontext wiederverwenden zu können, definieren Sie den entsprechenden Typ in einer separaten Quelldatei. Als Vorlage haben Sie die *Headerdatei* mit dem Dateinamen `queueelement.h` erhalten, die bereits in Ihrem Projekt vorhanden ist.

Die Datei sollte unbedingt einen Schutz gegen mehrfache Inklusion in der Form



Definieren Sie in der Datei `queueelement.h` eine Klasse `QueueElement` für ein einzelnes Listenelement.



Fügen Sie einen Schutz gegen Mehrfachinklusion in die Headerdatei ein.

```
#ifndef PROTECTOR_STRING
#define PROTECTOR_STRING
```

```
/* ...Restlicher Inhalt der Datei... */
```

```
#endif
```

enthalten, wobei `PROTECTOR_STRING` einer üblichen Konvention nach identisch mit dem Dateinamen (in Großbuchstaben und mit `_` statt des Punktes) sein sollte.

Der Sinn dieses Präprozessorkonstrukts besteht darin, dass im Fall einer versehentlichen oder unvermeidlichen Mehrfacheinbindung einer `.h`-Datei der Inhalt nur einmal vom Compiler verarbeitet wird, da es sonst zu Mehrfachdeklarationen und damit zu Fehlern kommt.

Wichtig hierbei ist es natürlich als `PROTECTOR_STRING` eine Zeichenkette zu wählen, welche *einmalig* im Projekt ist. Keinesfalls dürfen zwei `.h`-Dateien innerhalb eines Projektes die selbe Zeichenkette verwenden. Auch darf kein Variablenname oder C-Schlüsselwort verwendet werden, da dies sonst entsprechend vom Präprozessor ersetzt würde.

QueueElement
+ Previous :QueueElement* + Next :QueueElement* + Label :char[LABELSIZE] + X :int + Y :int + Z :int
+ QueueElement() + QueueElement(Label :const char* X :int, Y :int, Z :int, Previous :QueueElement* , Next :QueueElement*) + print() :void + getNext() :QueueElement* + getPrevious() :QueueElement*

Jetzt können Sie in dieser Datei die Klasse `QueueElement` definieren. Diese soll die Datenelemente entsprechend der nebenstehenden Abbildung (einem UML-Klassendiagramm) enthalten.

Neben den beiden Pointern zur Referenzierung des Vorgänger-Elements (`Previous`) und des Nachfolge-Elements (`Next`) sind noch die Nutzdaten in Gestalt des Arrays `Label` zu Speicherung einer kurzen Zeichenkette sowie drei Koordinatenkomponenten eines Punkts im dreidimensionalen Raum (`X`,

`Y` und `Z`) vorgesehen.

Hinweis: Die Klasse enthält auch sogenannte Konstruktoren. Stellen Sie sicher, dass Sie diese speziellen Methoden korrekt in der Headerdatei deklarieren.

Hinweis: Die Definition der globalen Konstante für die maximale Zeichenkettenlänge `LABELSIZE` ist bereits in Ihrer Dateivorlage enthalten.

Hinweis: Alle Daten- und Funktionselemente der Klasse sollen als **public** definiert werden.

Definieren Sie in Ihrer Klasse auch die aufgelisteten Methoden der Klasse. Die Funktionsbeschreibung dieser Methoden folgen dann in der nächsten Teilaufgabe.

Recherche ☕☕

Definition einer neuen Klasse

Recherche ☕

Implementierung von Konstruktoren

Recherche ☕

Defaultwerte für Funktionsparameter

Wichtig: Im UML-Diagramm nicht dargestellt ist, dass die beiden Parameter `Previous` und `Next` als Defaultwert den Wert `nullptr` erhalten sollen.

Hinweis: Geben Sie bei den Methoden nur den Prototyp an. Die Implementierung erfolgt dann in der nächsten Teilaufgabe.

Übersetzen Sie jetzt das Projekt. Es sollte sich ohne Fehler und Warnungen übersetzen lassen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie Ihre Element-Definition.

2.3 Erstellen der Methoden der Klasse `QueueElement`

Für die Arbeit mit den Elementen wird nun noch die Implementierung der verschiedenen Methoden benötigt. Deren Implementierung erfolgt in einer eigenen `.cpp`-Datei.

Hierfür erstellen Sie zunächst eine neue `.cpp`-Datei mit Namen `queueelement.cpp` und fügen Sie eine `#include`-Direktive für die oben bearbeitete `.h`-Datei hinzu.

Diese `.cpp`-Datei benötigt keinen Schutz gegen Mehrfachinklusion, da `.cpp`-Dateien niemals inkludiert werden dürfen. Sie werden vielmehr als voneinander syntaktisch unabhängige modulare Teile des Projektes automatisch von der Entwicklungsumgebung einzeln dem Compiler übergeben und die entstandenen Übersetzungsergebnisse im Anschluss vom Linker zu einem ausführbaren Programm zusammengesetzt. Aber das kennen Sie ja schon aus PIN1.

Implementieren Sie nun die Methoden der Klasse `QueueElement` in dieser neuen Datei gemäß folgender Beschreibung:

Hinweis: Da die Methoden außerhalb der `class`-Definition implementiert werden, müssen Sie den Namespace-Operator `::` verwenden, um dem Compiler mitzuteilen, dass Ihre Implementierung innerhalb des Namensraums der Klasse, hier `QueueElement`, erfolgen soll. Beispiel:

```
myclass::myMethod(thetype theparameter)
```

`QueueElement()`: Dieser Konstruktor ohne Parameter soll alle Elemente des Objekts initialisieren. Dabei sollen die beiden Pointer mit `nullptr` initialisiert werden, Zeichenkette als Leere Zeichenkette und die Koordinatenwerte mit dem `int`-Wert `Null`.

Wichtig: Verwenden Sie immer soweit technisch möglich die Initialisierersliste des Konstruktors für die Umsetzung der Initialisierungen!



Erzeugen Sie eine neue `.cpp`-Datei passend zur vorhandenen `.h`-Datei `queueelement.h` für die Implementierung der Methoden.



Erstellen Sie die Methoden der Klasse `QueueElement`.

Recherche ☕

Implementierung der Methoden außerhalb der Klassendefinition

Recherche ☕

Ausgabe auf `cout`

print(): Diese Methode gibt die Datenelemente des Objekts auf `cout` aus. Die Ausgabe soll dabei die Form (Beispiel!)
`QueueElement: 'labeltext' (x, y, z)`
 haben, wobei natürlich die korrekten Werte für `Label`, `X`, `Y` und `Z` auszugeben sind.

getNext(): Diese Funktion liefert den Wert des Datenelements `Next` zurück.

getPrevious(): Diese Funktion liefert den Wert des Datenelements `Previous` zurück.

Nun können Sie Ihr Programm testen.

Ihre Dozenten haben für Sie entsprechende Testfälle implementiert, um Ihnen die ersten Tests zu erleichtern. Diese sind derzeit noch ausgeblendet, da bisher die für den Test benötigten Elemente noch nicht vorhanden waren.

Nun können Sie die erste Stufe an Tests aktivieren, indem Sie in der Datei `testlevel.h` den Wert der Präprozessorkonstante `test_level` auf den Wert 1 erhöhen.

Übersetzen Sie jetzt das Projekt und prüfen Sie die Ausgabe. Das Programm muss sich ohne Warnungen übersetzen lassen und die implementierten Tests ohne Fehler ausführen.

Sollte das Programm nicht wie vorgesehen arbeiten, beheben Sie *jetzt* eventuelle Fehler bevor Sie mit der nächsten Teilaufgabe fortfahren.



Testen Sie Ihre bisherigen Implementierungen mit dem Testlevel 1.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

2.4 Erstellen des zweiten Konstruktors der Klasse `QueueElement`

Dieser Konstruktor

```
QueueElement(const char* Label, int X,  
int Y, int Z, QueueElement* Previous,  
QueueElement* Next)
```

soll ebenfalls alle Elemente des Objekts initialisieren. Dabei sollen die Datenelemente mit den Werten der passenden Parameter initialisiert werden.

Wichtig: Bis auf die Zeichenkette sollten Sie wieder alle Initialisierungen mit der Initialisiererliste umsetzen.

Bei der Zeichenkette ist etwas mehr Aufwand zu betreiben. Hier müssen Sie, wie üblich, mittels `strcpy` den Inhalt der Zeichenkette aus dem Funktionsparameter in das Array `Label` kopieren.

Prüfen Sie **vor** dem Kopiervorgang, ob die Zeichenkette nicht zu lang ist und führen Sie `strcpy` nur aus, falls die Zeichenkette kürzer ist als das Array `Label`.

Falls nicht ausreichend Platz vorhanden ist, kopieren Sie die Zeichenkette unter Verwendung von `strncpy`, welches ein zusätzliches drittes Argument besitzt, mit dem man die maximale zu kopierende Zeichenzahl festlegen kann.

Wichtig: Bei Verwendung von `strncpy` ist nicht sichergestellt, dass die Kopie am Ende eine binäre Null als Endekennzeichen enthält. Daher müssen Sie unbedingt an die letzte Position des Arrays `Label` eine binäre Null schreiben.

Falls Sie den extrem wichtigen Hinweis gerade eben nicht gelesen, sondern übersprungen haben, lesen Sie ihn jetzt! Wirklich!

Nun ist es an der Zeit, etwas kompliziertere Tests durchzuführen.

Sie können die nächste Stufe an Tests aktivieren, indem Sie in der Datei `testlevel.h` den Wert der Präprozessorkonstante `test_level` auf den Wert 2 erhöhen.

Übersetzen Sie jetzt das Projekt und prüfen Sie die Ausgabe. Das Programm muss sich ohne Warnungen übersetzen lassen und die implementierten Tests ohne Fehler ausführen.

Sollte das Programm nicht wie vorgesehen arbeiten, beheben Sie *jetzt* eventuelle Fehler bevor Sie mit der nächsten Teilaufgabe fortfahren.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie Ihre bisherigen Implementierungen mit dem Testlevel 2.

2.5 Definition der Klasse Queue

Nun kann die Klasse zur Verwaltung der Queue implementiert werden. Beginnen Sie damit, die Definition der Klasse in eine separate Datei `queue.h` zu schreiben. Die Datei wurde aus Komfortgründen bereits ohne Inhalt für Sie angelegt.



Definieren Sie eine Klasse `Queue` in einer eigenen `.h`-Datei.

Queue
+ FirstElement :QueueElement* + LastElement :QueueElement*
+ Queue() + Queue(InitialElement :QueueElement*) + ~Queue() + getFirstElement() :QueueElement* + getLastElement() :QueueElement* + push(QueueElement* NewItem) :void + pop() :void + insert(QueueElement* NewItem, unsigned Position) :void + isEmpty() :bool + clear() :void + size() :unsigned + print() :void

Damit Sie in dieser .h-Datei den Datentyp `QueueElement` verwenden können, müssen Sie eine Kurzdeklaration der Klasse angeben. Dies funktioniert analog zur Definition eines Funktionsprototypen: Statt der definierenden geschweiften Klammer setzen Sie lediglich einen Strichpunkt:

```
class myclass;
```

Nachdem Sie die Klasse definiert haben, erzeugen Sie eine passende .cpp-Datei für die Implementierung der Element-

funktionen.

Darin implementieren Sie nun folgende Funktionen:

Queue(): Initialisiert alle Datenelemente des Objekts mit dem Wert `nullptr`.

Hinweis: Verwenden Sie die Initialisiererliste!

Queue(QueueElement* InitialElement): Initialisiert alle Datenelemente des Objekts. Dabei soll das als Parameter übergebene Objekt als erstes Listenelement aufgenommen werden.

Hinweis: Verwenden Sie die Initialisiererliste!

Wichtig: Verwenden Sie hier keine anderen Methoden der Klasse, die Sie noch nicht implementiert haben!

getFirstElement(): Liefert einen Pointer auf das erste Element der Liste zurück (oder einen `nullptr` falls die Liste leer ist).

getLastElement(): Liefert einen Pointer auf das letzte Element der Liste zurück (oder einen `nullptr` falls die Liste leer ist).

size(): Diese Methode soll die verkettete Liste durchlaufen und dabei zählen, wie viele `QueueElement`-Objekte in der Liste enthalten sind. Das Ergebnis soll zurückgegeben werden.

Hinweis: Natürlich soll die Methode auch bei einer leeren Liste korrekt arbeiten.

Wichtig: Die Liste darf durch diese Methode nicht verändert werden.

isEmpty(): Diese Methode soll ermitteln, ob die Queue leer ist und das Ergebnis als Bool'schen Wert (Datentyp `bool`) zurückliefern.

Wichtig: Die Liste darf durch diese Methode nicht verändert werden.

print(): Gibt das Objekt auf `cout` aus. Die Ausgabe soll dabei folgende Form haben (hier am Beispiel einer Liste mit drei Elementen):

```

Queue:
<<>
<<0: QueueElement: 'Item Label #3' (18, 81, 1000)
<<1: QueueElement: 'Item Label #2' (16, 64, 1111)
<<2: QueueElement: 'Item Label #1' (14, 49, 1250)
<<!
```

Hinweis: Für die Ausgabe der einzelnen Listenelemente beider Ausgabe der Gesamtliste rufen Sie natürlich die von Ihnen bereits entwickelte Methode `print` der Klasse `QueueElement` auf.

Testen Sie nun den ersten Teil Ihrer Implementierung von `Queue`, indem Sie in der Datei `testlevel.h` den Wert der Präprozessorkonstante `test_level` auf den Wert 3 erhöhen.

Übersetzen Sie jetzt das Projekt und prüfen Sie die Ausgabe. Das Programm muss sich ohne Warnungen übersetzen lassen und die implementierten Tests ohne Fehler ausführen.

Sollte das Programm nicht wie vorgesehen arbeiten, beheben Sie *jetzt* eventuelle Fehler bevor Sie mit der nächsten Teilaufgabe fortfahren.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie Ihre bisherigen Implementierungen mit dem Testlevel 3.

2.6 Implementierung der restlichen Methoden der Klasse Queue

Nun benötigen Sie auch noch Methoden, um Elemente in die Queue vorne einzufügen und diese am Ende der Queue auch wieder zu entnehmen.

2.6.1 Elemente in die Queue einfügen

Überlegen Sie sich (zum Beispiel anhand einer Skizze), welche Vorgänge ablaufen müssen, um ein neues Objekt des Typs `QueueElement`, das per Pointer-Argument an die Funktion übergeben wird, am **Anfang** der Queue in die doppelt verkettete Liste **einzuhängen**.

Hinweis: Beachten Sie, dass Ihre Strategie immer funktionieren muss, also insbesondere bei Listen mit keinem, einem oder vielen Elementen. Gehen Sie bei dieser Überlegung möglichst sorgfältig vor. Sie ersparen sich damit zeitraubende Fehlersuche im weiteren Verlauf!

Frage: Werfen Sie Ihre Überlegungen nicht weg, sondern bewahren Sie sie in Ihrem Praktikumstagebuch auf, damit Sie sie bei der Aufgabenabgabe vorzeigen können. Das erleichtert die Beantwortung der Fragen bei der Abgabe.



Entwerfen Sie eine Methode zum Einfügen neuer Elemente am Anfang der Queue.

Implementieren Sie nun entsprechend Ihrer Überlegungen die Methode `Queue:push`.

2.6.2 Elemente aus der Queue entnehmen

Überlegen Sie sich (zum Beispiel anhand einer Skizze), welche Vorgänge ablaufen müssen, um ein Objekt am Ende der Queue zu **entfernen**. Dabei soll das betreffende Objekt **gelöscht**, also der belegte Speicher freigegeben werden.

Hinweis: Beachten Sie, dass Ihre Strategie immer funktionieren muss, also insbesondere bei Listen mit keinem, einem oder vielen Elementen. Gehen Sie bei dieser Überlegung möglichst sorgfältig vor. Sie ersparen sich damit zeitraubende Fehlersuche im weiteren Verlauf!

Frage: Werfen Sie Ihre Überlegungen nicht weg, sondern bewahren Sie sie in Ihrem Praktikumbuch auf, damit Sie sie bei der Aufgabenabgabe vorzeigen können. Das erleichtert die Beantwortung der Fragen bei der Abgabe.

Implementieren Sie nun entsprechend Ihrer Überlegungen die Methode `Queue:pop`.

Testen Sie nun den ersten Teil Ihrer Implementierung von `Queue`, indem Sie in der Datei `testlevel.h` den Wert der Präprozessorkonstante `test_level` auf den Wert 4 erhöhen.

Übersetzen Sie jetzt das Projekt und prüfen Sie die Ausgabe. Das Programm muss sich ohne Warnungen übersetzen lassen und die implementierten Tests ohne Fehler ausführen.

Sollte das Programm nicht wie vorgesehen arbeiten, beheben Sie *jetzt* eventuelle Fehler bevor Sie mit der nächsten Teilaufgabe fortfahren.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

2.7 Leeren der Queue

Mit Hilfe der nun vorhandenen Methoden, lassen sich die verbleibenden sehr einfach implementieren:

clear Diese Methode soll alle `QueueElement`-Objekte aus der Queue entfernen, so dass diese leer ist (kein Element mehr enthält).



Entwerfen Sie eine Methode zum Entfernen neuer Elemente am Ende der Queue.



Testen Sie Ihre bisherigen Implementierungen mit dem Testlevel 4.



Implementieren Sie den Destruktor sowie `Queue::clear()`.

Recherche ☕

Dynamisches Anlegen und Löschen von Objekten

Recherche ☕

Destruktoren

~Queue Der Destruktor soll alle durch das `Queue`-Objekt verwalteten Objekte löschen, damit das Objekt (von entsprechenden Verpflichtungen befreit) gelöscht werden kann.

Wichtig: Zur Erinnerung: Destruktoren löschen **niemals** Elemente des Objekts. Es werden ausschließlich vom Objekt verwaltete **externe** Objekte gelöscht, die **Eigentum** des Objekts sind.

Testen Sie nun den ersten Teil Ihrer Implementierung von `Queue`, indem Sie in der Datei `testlevel.h` den Wert der Präprozessorkonstante `test_level` auf den Wert 5 erhöhen.

Übersetzen Sie jetzt das Projekt und prüfen Sie die Ausgabe. Das Programm muss sich ohne Warnungen übersetzen lassen und die implementierten Tests ohne Fehler ausführen.

Sollte das Programm nicht wie vorgesehen arbeiten, beheben Sie *jetzt* eventuelle Fehler bevor Sie mit der nächsten Teilaufgabe fortfahren.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie Ihre bisherigen Implementierungen mit dem Testlevel 5.

2.8 Einfügen an beliebiger Position in die Queue

Manchmal ist es praktisch, in die Queue an beliebiger Stelle neue Elemente einzufügen (zum Beispiel einen hochprioren Druckauftrag oder ein Datenpaket für Audio-Streaming).

Hierfür soll die Methode `insert` bereitgestellt werden.

2.8.1 Entwickeln der Methode `insert`

Die Methode erhält als Parameter das einzufügende Objekt (per Pointer-Argument wie bei `push`) und einen ganzzahligen Index, der die Position angibt, an der das Objekt einzufügen ist.

Dabei soll ein Index von 0 ein Einfügen am Anfang der Queue bewirken, 1 ein Einfügen nach dem ersten Element usw. Ist der übergebene Wert des Index größer als die vorhandene Anzahl an Elementen in der Queue, so soll am Ende der Queue angehängt werden.

Überlegen Sie sich in bewährter Weise, welche Aktionen zum Einhängen in die Queue erforderlich sind. Beachten Sie, dass Ihre Funktion in allen Fällen korrekt funktionieren muss. Insbesondere auch bei leerer Queue, einer Queue mit einem oder vielen Elementen.

Wichtig: Überlegen Sie sich gründlich, welche Fälle es gibt und welche Schritte in diesen Fällen jeweils zu erledigen sind. Wenn



Entwickeln Sie die Methode `insert`.

Sie hier schludern, werden sie später viel Zeit auf die Fehlersuche verwenden!

2.8.2 Test von `Queue::insert`



Entwickeln Sie die Tests für die Methode `insert`.

Für die Methode `Queue::insert` haben Ihre Betreuer keinen Test implementiert.

Entwickeln Sie Tests für alle relevanten Fälle (Leere Liste, Liste mit einem Element, Liste mit mehreren Elementen und Einfügen am Anfang, in der Mitte und am Ende).

Gehen Sie dabei so vor, dass Sie eine neue Liste zu Testzwecken bauen und an dieser dann Einfügungen vornehmen.

Hinweis: Es reicht, wenn Sie die Ergebnisse nach Ausgabe mit `print` auf dem Bildschirm prüfen. Sie müssen keine Testauswertung mit dem `test`-Makro vornehmen.

Schreiben Sie Ihre Tests an diese Stelle der Datei `main.cpp`:

```
#if test_level >=6
    // insert tests for insert method here:

    std::cout << std::endl;
    std::cout << std::endl << "Test level 6 completed!"
        << std::endl << std::endl;
#endif
```



Testen Sie Ihre bisherigen Implementierungen mit dem Testlevel 6.

Testen Sie nun Ihre Implementierung, indem Sie in der Datei `testlevel.h` den Wert der Präprozessorkonstante `test_level` auf den Wert 6 erhöhen und damit Ihre selbst entwickelten Testfälle aktivieren.

Übersetzen Sie jetzt das Projekt und prüfen Sie die Ausgabe. Das Programm muss sich ohne Warnungen übersetzen lassen und die implementierten Tests ohne Fehler ausführen.

Sollte das Programm nicht wie vorgesehen arbeiten, beheben Sie eventuelle Fehler.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.